Software Architecture and Design Specification

Project: Hostel Management System (HMS)

Authors: Team 3 - G.V.N. Sreeharsha (PES1UG24CS163), Chennamraju Vaishnavi (PES1UG24CS129), Devansh Gupta (PES1UG24CS145)

Date: 07-10-2026

Status: Submitted for Part-1

Revision History

Version	Date	Contributor	Revision / Contribution
1.0	05-10-2026	G.V.N. Sreeharsha	Initial architecture and design baseline: components, layered pattern, data model, API design and security architecture.
1.1	07-10-2026	Chennamraju Vaishnavi	Reviewed design from the testing perspective; checked API, error handling and sequence diagrams against test cases; added the payment status report endpoint to API-R1.
1.2	07-10-2026	Devansh Gupta	Reviewed architecture and security design against the SRS; added the fee_amount field to the Room entity and API-H2, updated document cross-references and added team number and team member details.

Approvals

Role	Name	Signature/Date
Project Guide		
Team Lead	G.V.N. Sreeharsha	
Architect	Devansh Gupta	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Hostel Management System (HMS). It explains how the requirements in the HMS SRS are realised by components, interfaces and data structures.

1.2 Scope

Covers all HMS features: authentication, student management, hostel/room/bed management, bed allocation (allocate, transfer, release), fee payment and receipts, occupancy report, student self-view, audit logging and the security architecture. Hardware, network and hosting set-up are outside the scope.

1.3 Audience

Developers, QA engineers (Test Plan authors), security reviewers, the project guide and future maintainers.

1.4 Definitions

HMS (Hostel Management System), SRS (Software Requirements Specification), STP (Software Test Plan), MVT (Model-View-Template), ORM (Object Relational Mapping), RBAC (Role Based Access Control), CSRF (Cross Site Request Forgery), TLS (Transport Layer Security), STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), ADR (Architecture Decision Record), API (Application Programming Interface).

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (components, pattern, technology, risks, traceability and security). Section 4 describes the design (data model, sequence diagrams, API, error handling, UX). Component IDs (C1 to C10), sequence diagram IDs (SD-1, SD-2) and API IDs (API-xx) defined here are used in the traceability tables and in Appendix C of the SRS.

2.2 Related Documents

HMS Software Requirements Specification, HMS Software Test Plan (which holds the test cases and the RTM).

3. Architecture

3.1 Goals & Constraints

Goals: correct allocation (a bed is never given to two students), secure handling of student data, responses within 2 seconds for 50 concurrent users, simple modular code that a student team can build and test within one semester.

Constraints: Python with Django and MySQL 8.0 (course requirement); web application only; completion within the semester; PEP 8 coding standard; no external system integration in v1.0.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Admin	Control over hostels, rooms, beds and user accounts; reliability
Warden	Fast student registration and allocation; no double allocation; easy payment entry
Student	Privacy of personal data; simple view of own allocation and payment status
Developers	Modularity, clear interfaces, testability
QA / Testers	Every requirement traceable to a component and testable
Project guide / evaluators	Completeness against IEEE format, security design, traceability

3.3 Component (UML) Diagram

Figure 1: Component diagram of HMS

3.4 Component Descriptions

ID	Component	Responsibility	Django app (planned)
C1	Web UI	Renders pages and forms, handles user input and output messages, role-based navigation	templates, views
C2	Auth & Access Control	Login/logout, password hashing, account lockout, session timeout, role checks (Admin/Warden/Student)	accounts
C3	Student Module	Register, search, update and deactivate students;	students
C4	Hostel, Room & Bed Module	Add/edit hostels and rooms, auto-create beds, list available beds	hostels
C5	Allocation Module	Allocate, transfer and release beds in atomic transactions; enforce business rules BR-1 to	allocations
C6	Payment & Receipt Module	Record payments, generate unique receipts (PDF)	payments
C7	Report Module	Occupancy report and CSV export	reports
C8	Audit & Logging	Write audit entries (user, action, record, time) for logins and data changes; application error	audit
C9	Data Access Layer	Django ORM models, parameterised queries, transactions, soft delete	models

ID	Component	Responsibility	Django app (planned)
C10	MySQL Database	Persistent storage (InnoDB)	MySQL 8.0

3.5 Chosen Architecture Pattern and Rationale

A layered architecture (presentation, application, data) implemented with Django's Model-View-Template (MVT) pattern is chosen. Each feature is a separate Django app, so teammates can own modules independently (product ownership) and test them separately.

Rationale: clear separation of concerns; all database access goes through one layer (C9), which makes SQL-injection prevention and transaction control easy to verify; Django's built-in authentication, CSRF protection and ORM directly support the security requirements.

Rejected alternatives: Microservices were rejected as too complex to build, deploy and test for a single-hostel system with a small team. A single-file monolith without modules was rejected because it would not support independent ownership or unit testing.

3.6 Technology Stack & Data Stores

Item	Choice
Language / framework	Python 3.11+, Django 5.x
Front end	Django templates, HTML, CSS, Bootstrap 5
Database	MySQL 8.0 (InnoDB) accessed through Django ORM (mysqlclient driver)
PDF receipts	reportlab
Security	Django auth (PBKDF2), CSRF middleware, session framework, TLS 1.2+ at the web server
Testing / tools	Django test runner, Selenium, Postman, JMeter, OWASP ZAP, GitHub, GitHub Issues
Documentation / diagrams	draw.io / PlantUML style UML, Swagger/OpenAPI style API tables

3.7 Risks & Mitigations

Risk	Mitigation
Two wardens allocate the same bed at the same time	Single transaction with row lock (SELECT ... FOR UPDATE) and a database unique constraint on active allocation per bed
Slow search with 5,000 students	Indexes on Student ID, name and hostel; pagination of lists
Data loss	Daily database backup; restore tested (TC-OPS-01)
Security flaws in custom code	Use Django's built-in security features; run OWASP ZAP scan; review against Section 3.9
Team members' modules do not integrate	Fixed interfaces in Section 4.3; shared models in C9; integration tests each sprint

3.8 Traceability to Requirements

Every requirement of the SRS is allocated to at least one component (Architecture Ref) and to a design element (Design Ref). This table is repeated in Appendix C of the SRS.

Req ID	Component(s)	Design reference
REQ-1	C1, C2	API-A1; Sec 4.1
REQ-2	C2	API-A1; Sec 4.4
REQ-3	C2	API-A2; Sec 3.9
REQ-4	C2	API-A3
REQ-5	C3	SD-1; API-S1
REQ-6	C3, C9	SD-1; API-S1
REQ-7	C3, C9	API-S2
REQ-8	C3	API-S3
REQ-9	C3, C5	API-S4
REQ-10	C4	API-H1
REQ-11	C4	API-H2
REQ-12	C4, C9	API-H2
REQ-13	C4	API-H3
REQ-14	C5, C9	SD-2; API-L1
REQ-15	C5, C9	SD-2; API-L1
REQ-16	C5	SD-2; API-L1
REQ-17	C5, C9	API-L2
REQ-18	C5	API-L3
REQ-19	C5	API-L4
REQ-20	C6	API-P1
REQ-21	C6	API-P2
REQ-22	C6	API-P3
REQ-23	C7	API-R1
REQ-24	C2, C3	API-M1
NFR-1	C1, C9, C10	Sec 4.1 (indexes, pagination)
NFR-2	C5, C9	SD-2 (single transaction)
NFR-3	C9, C10	Sec 4.1 (indexes)
NFR-4	C10	Sec 4.4 (backup job)
NFR-5	C9	Sec 4.1 (soft delete)
NFR-6	C2	Sec 3.9
NFR-7	C2	Sec 3.9; Sec 4.3
NFR-8	C2, C3	API-M1; Sec 3.9
NFR-9	C2	API-A2; Sec 3.9
NFR-10	C1, C9	Sec 3.9; Sec 4.4
NFR-11	C8	Sec 4.4
NFR-12	C2, C1	API-A1; Sec 3.9

3.9 Security Architecture

Trust boundaries: (1) Browser to web server (untrusted network, protected by HTTPS); (2) Web server to application (every request is authenticated and authorised in C2); (3) Application to database (only C9 connects, using a least-privilege database account).

Threat modelling (STRIDE):

Threat	Example in HMS	Control (component) / requirement
Spoofing	Attacker guesses a warden's password	PBKDF2 hashed passwords, password policy, lockout after 5 failures (C2) - NFR-6, NFR-12
Tampering	Altering allocation or payment data, forged form submission	CSRF tokens, server-side validation, role checks, atomic transactions, payments not editable (C1, C2, C5, C6) - NFR
Repudiation	Warden denies releasing a bed	Audit log with user, action, record, timestamp (C8) - NFR-11
Information disclosure	Student reads another student's data; data sniffed on network	Own-data checks, RBAC, HTTPS TLS 1.2+, no sensitive data in logs or errors (C2, C3) - NFR-7, NFR-8, NFR-13
Denial of service	Repeated login attempts, heavy searches	Lockout, pagination and indexed queries (C2, C9) - NFR-1, NFR-12
Elevation of privilege	Warden opens admin URLs directly	Role check on every request, HTTP 403 (C2); only Admin manages users and hostels - NFR-7, BR-4

Security controls summary:

- Authentication: Django auth with PBKDF2 salted hashes; session timeout 15 minutes; HttpOnly and Secure cookies.
- Authorisation: a role decorator / middleware in C2 is applied to every view; default is deny.
- Input handling: Django forms for validation; ORM only (no string-built SQL); templates auto-escape output.
- Transport: HTTPS only; HTTP redirects to HTTPS.
- Accountability: all changes recorded in the audit table by C8.
- Database: least-privilege account; backups stored separately from the server.

Security objectives SO-1 to SO-4 of the SRS are met by the controls above: SO-1 (authentication, RBAC), SO-2 (validation, transactions, audit), SO-3 (own-data checks, HTTPS), SO-4 (lockout, backups).

4. Design

4.1 Design Overview

HMS is designed as layered Django apps. Views call service functions in the module apps; the service functions use ORM models and enclose multi-step changes in database transactions. Business rules BR-1 to BR-6 are enforced both in code and by database constraints.

Main data entities:

Entity	Key attributes	Relationships
User	username, password hash, role, is_active, failed_attempts	A Student user links to one Student

Entity	Key attributes	Relationships
Student	student_id (unique), name, dob, gender, course, year, mobile, email, guardian details, status	Has many Allocations and Payments
Hostel	name (unique), type (Boys/Girls)	Has many Rooms
Room	hostel, room_number (unique per hostel), floor, room_type, capacity (1–4), fee_amount	Has many Beds
Bed	room, bed_number, status	Has many Allocations over time
Allocation	student, bed, allocated_on, vacated_on (null = active)	Unique constraint: one active allocation per bed and per student
Payment	student, receipt_no (unique), amount, date, mode, reference	Belongs to Student; never edited
AuditLog	user, action, record, timestamp	Written by C8

Performance design: indexes on student_id, name, hostel and bed status; paginated lists (25 rows per page); allocation done in one short transaction.

Soft delete: students and users are marked inactive, never removed (NFR-5).

4.2 UML Sequence Diagrams

Two flows specific to HMS are modelled: SD-1 student registration and SD-2 bed allocation.

Figure 2: SD-1 Student Registration

Figure 3: SD-2 Bed Allocation

4.3 API Design

Interfaces are defined for all components. Each module exposes its operations as HTTP endpoints (rendered pages for the browser, and JSON responses that can be exercised with Postman). All endpoints except login require an authenticated session, a CSRF token for state-changing methods, and the roles listed. Common error format is given in Section 4.4.

4.3.1 Auth & Access Control (C2) - roles: all

ID	Method	Endpoint	Request	Success response	Errors
API-A1	POST	/auth/login/	{username, password}	200 {role, redirect}	401 invalid credentials; 423 account locked
API-A2	POST	/auth/logout/	-	204 No Content	401 not logged in
API-A3	POST / PATCH	/users/ and /users/{id}/	{username, password, role} / {is_active, role}	201 {id, role} / 200	400 invalid; 403 not Admin; 409 duplicate username

4.3.2 Student Module (C3) - roles: Admin, Warden (API-M1: Student)

ID	Method	Endpoint	Request	Success response	Errors
API-S1	POST	/students/	{student_id, name, dob, gender, course, year, mobile, email, guardian_name	201 {id, student_id, name}	400 field errors; 403; 409 Student ID already registered
API-S2	GET	/students/?q=&hostel=&page=	query parameters	200 {results[], page, total}	403

ID	Method	Endpoint	Request	Success response	Errors
API-S3	PATCH	/students/{id}/	{mobile, email, course, year, guardian..., address}	200 {student}	400; 403; 404; 422 student_id cannot be changed
API-S4	POST	/students/{id}/ deactivate/	-	200 {status: Inactive}	403; 404; 409 student has active allocation
API-M1	GET	/me/	-	200 {profile, allocation, payment_status}	401; 403 (only own data returned)

4.3.3 Hostel, Room & Bed Module (C4) - roles: Admin (write), Warden (read)

ID	Method	Endpoint	Request	Success response	Errors
API-H1	POST	/hostels/	{name, type}	201 {id}	400; 403; 409 duplicate name
API-H2	POST	/hostels/{id}/rooms/	{room_number, floor, room_type, capacity}	201 {room_id, beds[]}	400 capacity not 1-4; 403; 404; 409 duplicate room
API-H3	GET	/beds/available/?hostel=&floor=&room_ty pe=	query parameters	200 {beds[]}	403

4.3.4 Allocation Module (C5) - roles: Admin, Warden

ID	Method	Endpoint	Request	Success response	Errors
API-L1	POST	/allocations/	{student_id, bed_id}	201 {allocation_id, allocated_on}	403; 404 student/bed; 409 bed occupied or student already allocated; 422 gender mismatch
API-L2	POST	/allocations/{id}/ transfer/	{new_bed_id}	200 {allocation_id, bed}	403; 404; 409 new bed occupied; 422 gender mismatch
API-L3	POST	/allocations/{id}/ release/	{vacate_date}	200 {status: Released}	400 invalid date; 403; 404; 409 already released
API-L4	GET	/students/{id}/ allocations/	-	200 {allocations[]}	403; 404

4.3.5 Payment (C6) and Report (C7) Modules - roles: Admin, Warden

ID	Method	Endpoint	Request	Success response	Errors
API-P1	POST	/payments/	{student_id, amount, date, mode, reference?}	201 {receipt_no}	400 amount not > 0 / bad mode; 403; 404
API-P2	GET	/payments/ {receipt_no}/ receipt.pdf	-	200 PDF file	403; 404
API-P3	GET	/students/{id}/ payment-status/	-	200 {status: Paid|Partial|Pending, paid, due}	403; 404
API-R1	GET	/reports/ occupancy/?hostel=&format=c	query parameters	200 {rows[]} or CSV file	403; 404

Example - POST /allocations/ (API-L1)

Request	Response
POST /allocations/ Content-Type: application/json X-CSRFToken: <token> { "student_id": "PES1UG24CS163", "bed_id": 42 }	201 Created { "allocation_id": 311, "allocated_on": "2026-10-05", "bed": "101-A" } Error example: 409 { "error": { "code": "BED_OCCUPIED", "message": "Bed 101-A is already occupied" } }

4.4 Error Handling, Logging & Monitoring

Standard error format: { "error": { "code": "<MACHINE_CODE>", "message": "<plain-language text>", "fields": {optional field errors} } }. Browser pages show the same message next to the form field or at the top of the page.

HTTP	Code	When	User message / behaviour
400	VALIDATION_ERROR	Missing or invalid input	Field-level messages; nothing is saved
401	AUTH_REQUIRED / INVALID_CREDENTIALS	Not logged in or wrong login	"Invalid username or password" (does not say which was wrong); redirect
403	ACCESS_DENIED	Role not allowed, CSRF failure	"Access denied"; no data changed; event logged
404	NOT_FOUND	Record does not exist	"Record not found"
409	DUPLICATE / BED_OCCUPIED / ALREADY_ALLOCATED / HAS_ACTIVE_ALLOCATION	Business rule conflict	Message names the conflict, for example "Student ID already registered"
422	GENDER_MISMATCH / IMMUTABLE_FIELD	Rule violation on valid input	Message explains the rule
423	ACCOUNT_LOCKED	5 failed logins	"Account locked for 15 minutes"
500	SERVER_ERROR	Unexpected failure	Generic message "Something went wrong"; transaction rolled back; details only in server log

- Transactions: allocation, transfer and release failure rolls back so data is never half-updated.
- Logging: application log (errors, with a request ID) and audit log (who did what and when). Passwords, session IDs and full personal details are never written to logs.
- Error pages never show stack traces, SQL or file paths (Django DEBUG is off in deployment).
- Monitoring: track login failure count, HTTP 4xx/5xx rate, average response time, backup job success, and database free space; review weekly during the project.

4.5 UX Design

- Role-based dashboard and navigation menu showing only permitted options.
- Forms with required-field markers (*), inline validation messages, and confirmation dialogs for release, transfer and deactivate actions.
- Bootstrap responsive layout, usable on laptop and mobile screens; readable font sizes and sufficient colour contrast.
- Allocation screen: select student, view filtered list of available beds, confirm - designed so a new warden completes it in under 5 minutes (quality attribute: Usability).
- Screens: Login, Dashboard, Student List/Form, Hostel/Room/Bed List, Allocation, Payment, Receipt, Reports, Staff Accounts, My Allocation (Student).

4.6 Open Issues & Next Steps

- Module ownership will be assigned in the sprint plan before implementation; the architecture supports independent ownership of C2 to C7.
- For v1.0, the fee amount will be maintained as a configurable value per room type and used by the Payment & Receipt Module to calculate Paid, Partial or Pending status.
- For v1.0, account lockout will be implemented in C2 using the existing failed_attempts and locked_until fields, with a 15-minute lockout after 5 consecutive failed attempts.
- Fill the Code File Ref column of SRS Appendix C as modules are implemented.
- Future enhancements: online payment gateway, e-mail notifications, mess management, visitor log.

5. Appendices

5.1 Glossary: HMS, SRS, STP, MVT, ORM, RBAC, CSRF, TLS, STRIDE, ADR, API, SD (Sequence Diagram), C1-C10 (components).

5.2 References: IEEE 42010 (Architecture description), IEEE Std 830 (SRS), OWASP ASVS 4.0, Django documentation 5.x, MySQL 8.0 Reference Manual, HMS SRS, HMS Software Test Plan.

5.3 Tools: draw.io / PlantUML style diagrams, Swagger/OpenAPI style tables, Postman, GitHub.